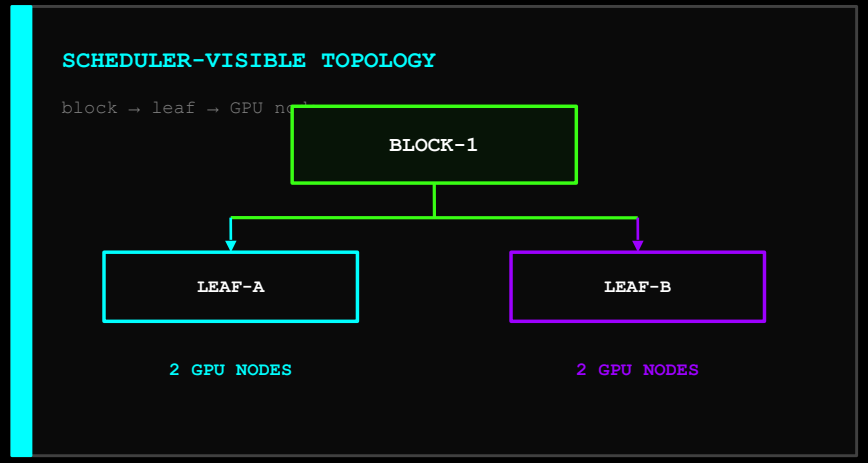
THE SCHEDULER HAS NEVER HEARD OF NVLINK

Topology-Aware GPU Placement on Kubernetes

speaker.identity

Neeraj Pandey

Co-founder & CTO · Lyntcube



THIS IS A CONTROL-PLANE LAB — NO GPU REQUIRED

LIVE ON EVERY LAPTOP

real Kubernetes decisions on fake capacity

- 5-node kind cluster
- fake extended GPU resources
- block / leaf / rack labels
- Kueue 0.19 Topology + ResourceFlavor
- required, impossible and preferred placement

REAL-WORLD EVIDENCE

shown, analyzed and taken home

- H100 NVLink / NVSwitch hierarchy
- InfiniBand NDR / HDR physics
- DCGM and application MFU
- nccl-tests benchmark methodology
- admission and topology-source architecture

Every GPU is fake. Every admission and scheduling decision is real.

BY 15:00, EVERY PARTICIPANT WILL HAVE BUILT THIS

01	EXPLAIN	**why topology-blind placement can waste 40–60% throughput**
02	EXPOSE	physical topology through an owned Node-label data contract
03	MODEL	block → leaf → hostname with Kueue Topology + ResourceFlavor
04	ENFORCE	whole-PodSet locality before any trainer starts
05	VALIDATE	topology intent at admission and prove outcomes with telemetry

The artifact is not YAML. It is a defensible placement control plane.

CENTRAL THESIS

GPU CAPACITY IS NOT ENOUGH.

PLACEMENT QUALITY IS A RESOURCE.

A GPU cluster has a topology graph, not merely an inventory.

decision.contract

quantity + locality + all-or-nothing admission + evidence

SAME 32 GPUS. TWO COMPLETELY DIFFERENT MODELS.

KUBERNETES RESOURCE ACCOUNTING

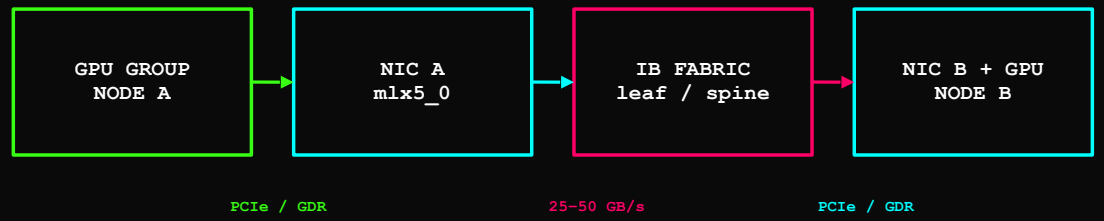
quantity and per-Pod feasibility

- node-a H100 ×8
- node-b H100 ×8
- node-c H100 ×8
- node-d H100 ×8

32 × nvidia.com/gpu

NCCL COMMUNICATION GRAPH

every edge has bandwidth, latency and contention



BOTTLENECK

collective throughput is bounded by the slowest path segment—not the GPU count

32 POSITIONS IN A GRAPH

A vanilla GPU request encodes quantity. It does not encode the path between peers.

THE UNIT ERROR THAT CHANGES THE ENTIRE ARGUMENT

PATH	BANDWIDTH	SCOPE	ORDER
H100 NVLink / NVSwitch	~900 GB/s	intra-node GPU ↔ GPU	01
PCIe Gen5 x16	~64 GB/s	GPU ↔ CPU / NIC	02
InfiniBand NDR	400 Gb/s ≈ 50 GB/s	current-gen inter-node	03
InfiniBand HDR	200 Gb/s ≈ 25 GB/s	prior-gen inter-node	04
cross-leaf / congested	lower still	extra spine hops	05

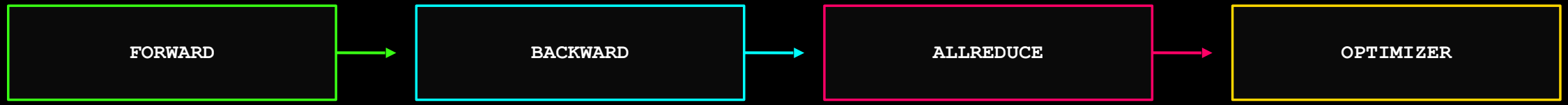
DO NOT SAY

"HDR is 200 GB/s" — HDR is 200 Gb/s per port, roughly 25 GB/s raw line rate.

GENERATION CHECK

600 GB/s is A100 NVLink. H100 fourth-generation NVLink is ~900 GB/s.

ALLREDUCE PUTS COMMUNICATION ON EVERY TRAINING STEP



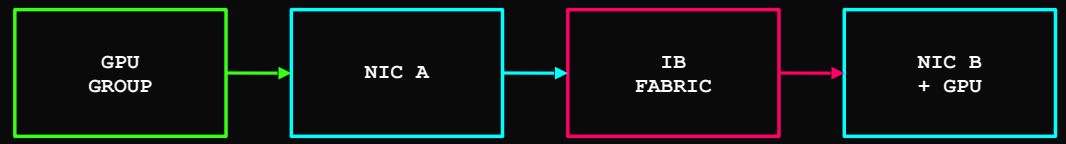
step-time.model

$$T_{step} = T_{compute} + T_{communication}$$

$$T_{communication} \approx f(\text{message size, collective algorithm, topology, congestion})$$

COLLECTIVE CRITICAL PATH

each rank advances at the slowest transfer boundary



PCIe / GDR

25–50 GB/s

SLOWEST EDGE SETS THE STEP

Ring and tree algorithms distribute traffic differently; bottleneck paths still constrain the collective.

THE END-TO-END RESPONSIBILITY MAP

NETWORK / DCIM
truth

PLATFORM
contract

KUEUE
domain

SCHEDULER
node

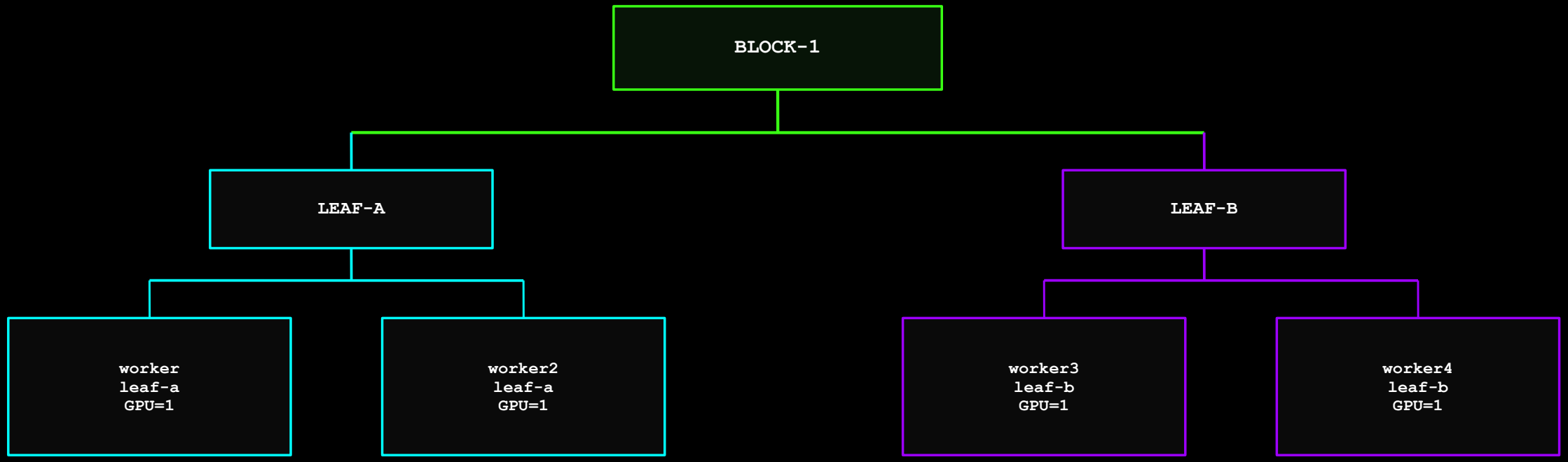
KUBELET
NUMA

• • • invariant

Every label has a source, every transition has an owner, every outcome has evidence.

Modern Kubernetes can enforce topology. It does not discover your physical fabric for you.

ONE BLOCK. TWO LEAVES. TWO GPU NODES PER LEAF.



MODEL
extended resources represent scheduler capacity

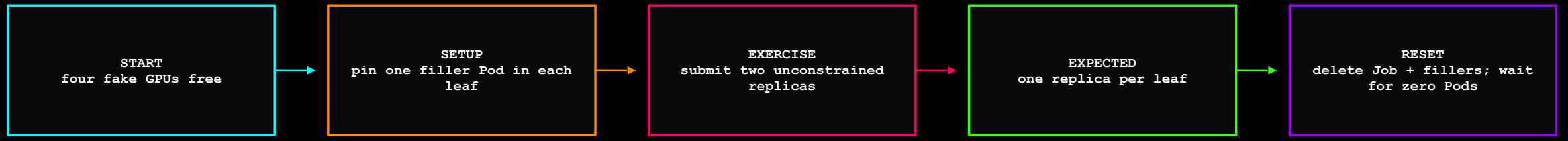
NOT A SIMULATOR
no CUDA, NCCL or bandwidth is emulated

REAL DECISION
Kueue and scheduler behavior is genuine

Fake hardware. Real control plane.

CODE → cluster-setup/kind-config.yaml · topology-pipeline/sample-inventory.json

EVERY LAB HAS A TRANSACTION BOUNDARY



terminal

```
01 make checkpoint-1
02 make lab1-demo
03 make lab1-reset
04 make verify-lab1
```

Cleanup is part of the experiment—not a courtesy after it.

DEMO → make checkpoint-1 · make lab1-demo · make lab1-reset

MAKE THE BAD PLACEMENT REPRODUCIBLE FOR ALL 40 LAPTOPS

DEFAULT SCHEDULER: VALID QUANTITY PLACEMENT · GUARANTEED CROSS-LEAF

Accidental good placement is a poor teaching dependency.

CODE → jobs/placement-fillers.yaml

LAB 1: THE JOB DECLARES QUANTITY — NOTHING ABOUT PEERS

jobs/training-unaware.yaml · relevant fields

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: training-unaware
  namespace: training
spec:
  parallelism: 2
  completions: 2
  template:
    spec:
      containers:
      - name: trainer
        image: registry.k8s.io/e2e-test-images/agnhost:2.53
        args: ["pause"]
        resources:
          requests:
            workshop.example.com/gpu: 1
          limits:
            workshop.example.com/gpu: 1
      restartPolicy: Never
```

Creates 2 trainer pods
Job is called training-unaware and runs in the training namespace.

SCHEDULER INPUT

what is actually encoded

- parallelism = 2
- GPU = 1 / Pod

MISSING CONTRACT

nothing asks for peer locality

- no queue
- no PodSet topology
- no leaf preference
- no all-or-nothing admission

The scheduler did exactly what the object asked.

CODE → jobs/training-unaware.yaml

A VALID PLACEMENT CAN STILL BE THE WRONG PLACEMENT

terminal · deterministic result

```
$ make lab1-demo

$ kubectl get pods -n training -o wide
trainer-0   Running   topology-lab-worker2
trainer-1   Running   topology-lab-worker4

$ kubectl get nodes -L workshop.example.com/leaf
topology-lab-worker2  leaf-a
topology-lab-worker4  leaf-b

PASS  two replicas · two leaves · zero scheduler errors
```

test oracle

scheduled=2 AND distinct_leaves=2 → topology-blind failure reproduced

KUBERNETES RESULT

2 Pods scheduled · 2 GPUs allocated · zero errors

TRAINING RESULT

communication crosses a fabric boundary that was absent from the request

ROOT CAUSE

locality was not represented as policy; there was nothing for the scheduler to optimize

No alert fires. The bill arrives later.

DEMO → make lab1-demo · scripts/verify-unaware-placement.sh

CHECKPOINT 1: RESET CAPACITY BEFORE KUEUE

terminal · checkpoint contract

```
$ make lab1-reset
$ make verify-lab1

STATE  nodes Ready              5 / 5
STATE  lab Pods                 0
STATE  fake GPUs allocatable    4 / 4

OK  worker  -> block-1 / leaf-a / GPU=1
OK  worker2 -> block-1 / leaf-a / GPU=1
OK  worker3 -> block-1 / leaf-b / GPU=1
OK  worker4 -> block-1 / leaf-b / GPU=1

CHECKPOINT 1 VERIFIED
```

recovery transaction

INVARIANTS

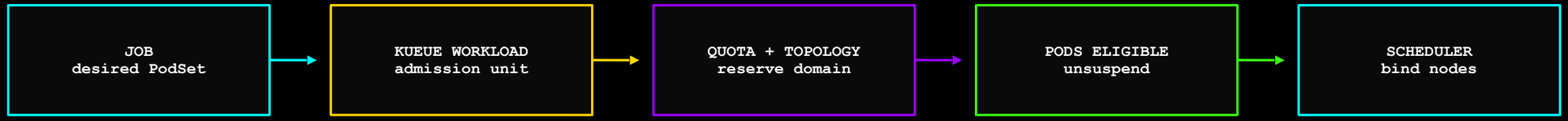
do not continue until all pass

- five nodes Ready
- labels match inventory
- capacity = 1
- allocatable = 1
- zero lab Pods

Recovery checkpoints turn forty unique laptops into one known state.

DEMO → make lab1-reset · scripts/verify-lab1.sh

KUEUE IS THE GATE IN FRONT OF THE SCHEDULER



KUEUE DECIDES

whether quota and a valid topology domain can be reserved for the whole PodSet

SCHEDULER DECIDES

which concrete nodes satisfy the admitted assignment and ordinary Pod constraints

• • • ownership.boundary

Kueue is not another scheduler. It controls eligibility before scheduling begins.

Admission prevents partial GPU allocation from becoming an expensive waiting room.

CODE → Makefile · scripts/checkpoint.sh

FOUR KUEUE OBJECTS — FOUR DIFFERENT CONTRACTS

01 TOPOLOGY

`spec.levels`

block → leaf → hostname

02 RESOURCEFLAVOR

`spec.topologyName`

eligible inventory

03 CLUSTERQUEUE

`spec.resourceGroups`

flavor-scoped quota

04 LOCALQUEUE

`spec.clusterQueue`

tenant entry point

• • • dependency contract

`Topology.name` → `ResourceFlavor.topologyName` → `ClusterQueue.flavors.name` → `LocalQueue.clusterQueue`

ADMISSION UNIT

The Job becomes a Workload; its PodSet is evaluated against quota and topology together.

NOT COHORT

A Cohort shares quota across ClusterQueues. It does not provide gang or topology placement.

Topology describes. ResourceFlavor attaches. ClusterQueue reserves. LocalQueue accepts.

CODE → kueue-config/{topology,resource-flavor,cluster-queue,local-queue}.yaml

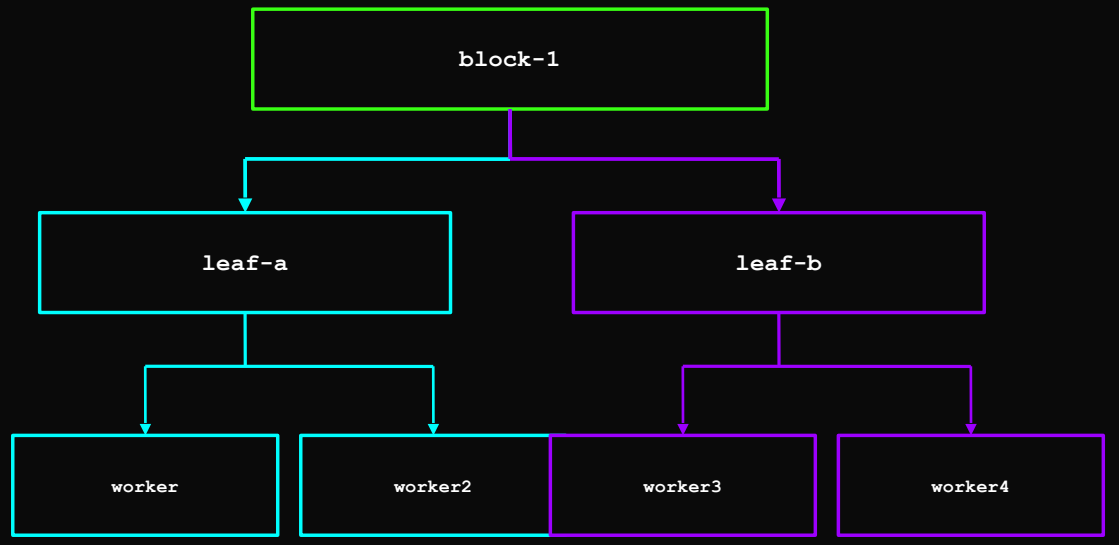
THE TOPOLOGY OBJECT TURNS LABELS INTO A HIERARCHY

kueue-config/topology.yaml

```yaml
apiVersion: kueue.x-k8s.io/v1beta2
kind: Topology
metadata:
  name: gpu-fabric
spec:
  levels:
  - nodeLabel: workshop.example.com/block
  - nodeLabel: workshop.example.com/leaf
  - nodeLabel: kubernetes.io/hostname
```

DOMAIN TREE

capacity is evaluated at each level



A topology level is a scheduling domain, not a decorative label.

CODE → kueue-config/topology.yaml

RESOURCEFLAVOR CONNECTS CAPACITY TO TOPOLOGY

kueue-config/resource-flavor.yaml

```yaml
apiVersion: kueue.x-k8s.io/v1beta2
kind: ResourceFlavor
metadata:
  name: h100-topology
spec:
  nodeLabels:
    workshop.example.com/gpu-node: "true"
  topologyName: gpu-fabric
```

SELECT

nodeLabels defines which Nodes belong to this capacity class

ATTACH

topologyName makes gpu-fabric available to TAS for this flavor

server-side verification

```
kubectl get resourceflavor h100-topology
-o jsonpath='{.spec.topologyName}'  →  gpu-fabric
```

IMMUTABILITY

Once topologyName is set, the entire ResourceFlavor spec is immutable: delete and recreate to change it.

FAIL CLOSED

A Node without gpu-node=true is ineligible for this flavor; it cannot silently enter the topology pool.

Topology models the graph. ResourceFlavor says which inventory participates.

CODE → kueue-config/resource-flavor.yaml

QUOTA AND TOPOLOGY MEET IN THE CLUSTERQUEUE

kueue-config/cluster-queue.yaml · relevant fields

```yaml
apiVersion: kueue.x-k8s.io/v1beta2
kind: ClusterQueue
metadata:
  name: gpu-training
spec:
  namespaceSelector:
    matchLabels:
      app.kubernetes.io/part-of: nvlink-topology-workshop
  resourceGroups:
  - coveredResources: [workshop.example.com/gpu]
    flavors:
    - name: h100-topology
      resources:
      - name: workshop.example.com/gpu
        nominalQuota: 4
```

kueue-config/local-queue.yaml

```yaml
apiVersion: kueue.x-k8s.io/v1beta2
kind: LocalQueue
metadata:
  name: gpu-queue
  namespace: training
spec:
  clusterQueue: gpu-training
```

quota tuple

resource=workshop.example.com/gpu · flavor=h100-topology · quota=4

SCOPE

Only namespaces labeled as part of this workshop may consume gpu-training quota.

CAPACITY CONTRACT

4 fake GPUs · one topology-aware flavor · workshop namespaces only

JOB CONTRACT

kueue.x-k8s.io/queue-name: gpu-queue routes the Job into admission.

CODE → kueue-config/cluster-queue.yaml · local-queue.yaml

CHECKPOINT 2: INSTALL FROM CACHE, THEN VERIFY THE CONTRACT

terminal · checkpoint contract

```
$ make checkpoint-2

OK  5 nodes Ready
OK  topology labels match inventory
OK  fake GPU allocatable = 4 / 4
OK  Kueue controller available
OK  Topology levels: block leaf hostname
OK  ResourceFlavor -> gpu-fabric
OK  ClusterQueue Active=True
OK  training/gpu-queue -> gpu-training

CHECKPOINT 2 VERIFIED
```

admission readiness gate

API served ∧ controller Ready ∧ flavor attached ∧ ClusterQueue Active

The participant's state is now known—not merely hoped for.

WHY THIS IS IDEMPOTENT

safe recovery during a live room

- reuses existing cluster
- reapplies labels
- repatches capacity
- installs local manifest
- reapplies API objects
- deletes stale lab workloads

DEMO → make checkpoint-2 · scripts/verify-lab2.sh

LAB 3 · 15 MIN

03

REQUIRE, BREAK, THEN RELAX LOCALITY

Observe the topology assignment—not just the final Pod nodes.

NEXT CODE → jobs/training-topology-{required,preferred}.yaml

ONE ANNOTATION CHANGES THE ADMISSION UNIT

jobs/training-topology-required.yaml · runnable core

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: training-topology-aware
  namespace: training
  labels:
    kueue.x-k8s.io/queue-name: gpu-queue
spec:
  parallelism: 2
  completions: 2
  completionMode: Indexed
  template:
    metadata:
      annotations:
        kueue.x-k8s.io/podset-required-topology:
          workshop.example.com/leaf
    spec:
      containers:
      - name: trainer
        image: registry.k8s.io/e2e-test-images/agnhost:2.53
        args: ["pause"]
        resources:
          requests:
            workshop.example.com/gpu: 1
          limits:
            workshop.example.com/gpu: 1
      restartPolicy: Never
```

admission invariant

QUEUE

Job becomes a Kueue Workload before Pods are eligible.

PODSET

parallelism=2 is considered as one capacity decision.

REQUIRED

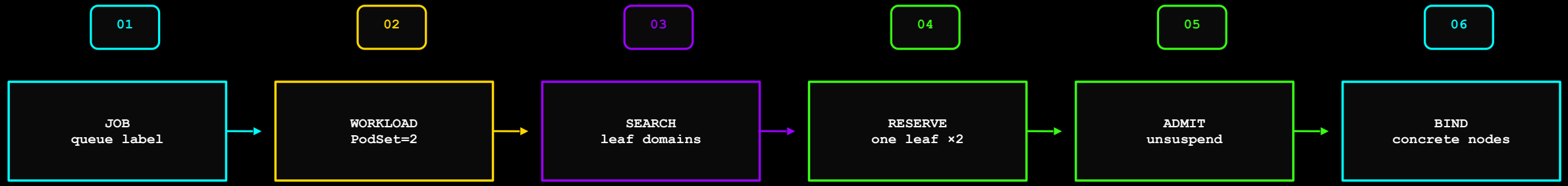
Both replicas must fit inside one leaf or the Workload waits.

INDEXED

Stable completion indexes make trainer identity deterministic.

CODE → jobs/training-topology-required.yaml

TAS RESERVES A DOMAIN BEFORE ANY TRAINER STARTS



CAPACITY SNAPSHOT

Kueue evaluates free capacity by topology domain

LEAF-A
2 GPUs free → candidate

LEAF-B
2 GPUs free → candidate

PERSISTED EVIDENCE

Workload.status.admission

`podSetAssignments[].topologyAssignment` → leaf-a OR leaf-b

The assignment is inspectable API state—not an inference from scheduler logs.

CODE → scripts/verify-lab3.sh

VERIFY THE DECISION AT TWO LAYERS

terminal · inspect persisted intent and outcome

```
$ make lab3-required

$ workload=$(kubectl get workloads -n training \
    -o jsonpath='{.items[0].metadata.name}')
$ kubectl get workload "$workload" -n training \
    -o
jsonpath='{.status.admission.podSetAssignments[0].topologyAssignment.domains}'
[{count: 2, values: [leaf-a]}]

$ kubectl get pods -n training \
    -l job-name=training-topology-aware -o wide
trainer-0  Running  topology-lab-worker
trainer-1  Running  topology-lab-worker2
```

automated consistency assertion

CONTROL-PLANE EVIDENCE

Workload status records the domain Kueue reserved for the PodSet.

DATA-PLANE EVIDENCE

Both Pod nodeNames map to the same leaf label.

AUTOMATED ASSERTION

verify-lab3.sh fails if the assignment and observed leaf count disagree.

Trust persisted intent and observed outcome—then compare them.

DEMO → make lab3-required · scripts/verify-lab3.sh

REQUIRED LOCALITY FAILS CLOSED

LEAF-A

capacity = 2

GPU GPU

NO THIRD SLOT

LEAF-B

capacity = 2

GPU GPU

NO THIRD SLOT

WORKLOAD

requires leaf locality

REPLICAS = 3

ADMITTED = FALSE

PODS CREATED = 0

terminal · assertion

```
make lab3-impossible  →  Workload: Pending  ·  Trainer Pods: 0
```

Waiting is cheaper than allocating a partial gang that cannot train.

CODE → jobs/training-impossible.yaml · make lab3-impossible

REQUIRED AND PREFERRED ARE DIFFERENT BUSINESS PROMISES

POLICY	SEARCH BEHAVIOR	NO LEAF FITS	OPERATOR PROMISE
REQUIRED	evaluate the requested leaf level	Workload waits	never cross this boundary
PREFERRED	try leaf, then widen upward	admits at block	optimize locality, preserve progress

REQUIRED

hard compliance / performance boundary

leaf-a × leaf-b × WAIT

PREFERRED

fallback policy explicitly accepted

leaf × → block ✓ → ADMIT

Fallback is policy. Silence is not a fallback policy.

COMPARE → jobs/training-impossible.yaml · training-topology-preferred.yaml

LAB 3C: CHANGE ONE WORD, CHANGE THE OUTCOME

manifest diff

```
template:
  metadata:
    annotations:
-     kueue.x-k8s.io/podset-required-topology:
+     kueue.x-k8s.io/podset-preferred-topology:
        workshop.example.com/leaf
```

terminal

```
make lab3-preferred

leaf-a: trainer replicas
leaf-b: trainer replica

Admitted=True
topology level widened to block
```

SAME DEMAND

3 replicas · 3 fake GPUs · one training Job

DIFFERENT POLICY

cross-leaf execution is now allowed when no leaf fits

OBSERVABLE

Workload topologyAssignment shows the widened domain

The API makes the queue-time versus locality tradeoff explicit.

CODE → jobs/training-topology-preferred.yaml · make lab3-preferred

CHECKPOINT 3: RESTORE A CLEAN TAS-READY CLUSTER

terminal · final recovery boundary

```
$ make checkpoint-3

RESTORE  topology inventory labels
RESTORE  fake GPU allocatable state
RESTORE  Kueue controller and API objects
DELETE   lab Jobs and filler Pods

STATE    Nodes Ready              5 / 5
STATE    GPUs allocatable         4 / 4
STATE    ClusterQueue Active      True
STATE    lab Pods                 0

CHECKPOINT 3 VERIFIED
```

clean-state predicate

ready=5 ∧ allocatable=4 ∧ Active=True ∧ labPods=0

OPTIONAL BRANCHES

only if the room is ahead

- inspect Workload YAML
- apply CEL policy
- change leaf capacity
- change required level
- discuss priority / fairness

Hands-on work ends in a known state; advanced concepts can now build on it safely.

DEMO → make checkpoint-3

PARTIAL PLACEMENT CAN LOOK BUSY WHILE DOING ZERO TRAINING

JOB REQUIRES FOUR TRAINERS

GANG CONTRACT

Can minCount members fit? YES → release the group. NO → release none.

Allocated does not mean productive.

NOTEBOOK → Phase 4 · partial-placement model

DO NOT COLLAPSE FOUR SCHEDULING CONCEPTS INTO 'GANG'

MECHANISM	QUESTION IT ANSWERS	OBJECT / API	NOT ITS JOB
Cohort	who may borrow quota?	Kueue Cohort	placement
Kueue Workload	what is admitted together?	kueue.x-k8s.io	node binding
TAS	which topology domain fits?	Topology + ResourceFlavor	NUMA alignment
PodGroup / gang	can minCount start together?	scheduling.k8s.io/v1beta1	fabric discovery

• • • rule

TAS chooses locality. Gang scheduling protects synchronized progress.

Orthogonal guarantees compose; they do not substitute for one another.

NOTEBOOK → Phase 4 · TAS / gang / Cohort comparison

KUEUE TAS ≠ KUBELET TOPOLOGY MANAGER

PROPERTY	KUEUE TAS	TOPOLOGY MANAGER
scope	cluster	one node
runs in	Kueue control plane	kubelet
topology unit	block / rack / leaf / hostname	NUMA domain
purpose	whole-PodSet domain locality	CPU / memory / device alignment
decision	which domain?	is this node-local allocation acceptable?

• • • KubeletConfiguration fields

```
topologyManagerPolicy: single-numa-node
topologyManagerScope: pod
```

COMPOSE

TAS chooses the node domain; Topology Manager protects CPU/GPU/NIC alignment inside the selected node.

NOTEBOOK → §13 node-local NUMA alignment

EVERY TOPOLOGY FACT NEEDS ONE SOURCE OF TRUTH

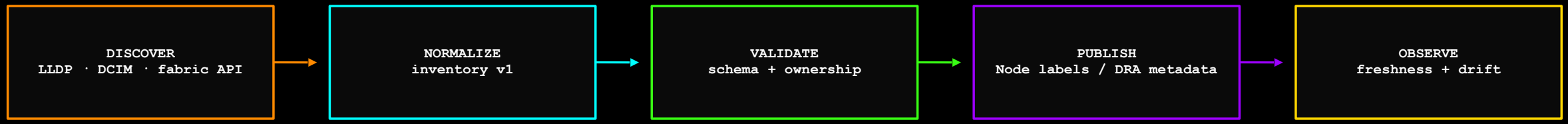
DATA	AUTHORITATIVE SOURCE	OWNER	KUBERNETES OUTPUT
node → rack	NetBox / DCIM	data-center operations	workshop.example.com/rack
node → leaf / block	LLDP + fabric manager	network engineering	leaf / block labels
NIC speed / state	fabric manager	network engineering	policy + health signal
GPU SKU / device health	GPU Operator / DRA	GPU platform	flavor / ResourceSlice
NUMA topology	kubelet / device layer	node platform	Topology Manager hints
job locality policy	ML platform	training platform	required / preferred annotation

BOUNDARY

Infrastructure owns facts. The ML platform owns placement policy. Kubernetes enforces the translated contract.

CODE → topology-pipeline/sample-inventory.json

THE SYNC CONTROLLER IS A RECONCILIATION LOOP



SOURCE UNAVAILABLE
retain last-known-good + mark stale

CONFLICTING SOURCES
do not guess; quarantine mapping

NODE MOVED LEAF
cordon/drain before relabel

LABEL DRIFT
controller reconciles desired state

A stale topology label is a silent scheduling bug with administrative confidence.

CODE → scripts/render-topology.py · inventory.schema.json

STATIC POLICY IN CEL. EXTERNAL STATE IN A WEBHOOK.

VALIDATINGADMISSIONPOLICY

built in · no service · no TLS

- queue label exists
- exactly one fallback policy
- topology level is approved
- namespace-scoped binding
- fast deterministic denial

VALIDATING WEBHOOK

only when external data is necessary

- tenant entitlement lookup
- DCIM/fabric policy lookup
- change-window validation
- organization-specific risk rules
- timeout + failurePolicy designed explicitly

SEPARATION

CEL checks object-local invariants. The webhook checks external policy. Neither performs placement.

Choose the smallest enforcement mechanism that can express the rule.

CODE → webhook/validating-admission-policy.yaml

OBSERVE THE PATH, THE HARDWARE, AND THE MODEL

CONTROL PLANE

WHAT WAS INTENDED

- queue wait
- topologyAssignment
- Pod → Node → leaf

HARDWARE PLANE

WHAT PATH RAN

- SM / tensor activity
- NVLink / PCIe bytes
- IB counters

APPLICATION PLANE

WHAT WORK COMPLETED

- step time
- tokens / second
- MFU + loss

• • • correlation key

topology identity + interconnect activity + model progress → placement diagnosis

A dashboard without topology identity cannot explain topology loss.

CODE → kubectl Workload/Pod evidence · observability/

DCGM MEASURES ACTIVITY. MFU MEASURES USEFUL PROGRESS.

SIGNAL	WHAT IT MEASURES	HOW TO USE IT
DCGM_FI_PROF_SM_ACTIVE	cycles with ≥1 warp assigned	activity, not efficiency
DCGM_FI_PROF_PIPE_TENSOR_ACTIVE	tensor/HMMA pipe activity	correlate with expected math
DCGM_FI_PROF_DRAM_ACTIVE	memory-interface activity	identify memory pressure
DCGM_FI_PROF_NVLINK_TX_BYTES DCGM_FI_PROF_NVLINK_RX_BYTES	intra-node link traffic	verify intended fast path
DCGM_FI_PROF_PCIE_TX_BYTES DCGM_FI_PROF_PCIE_RX_BYTES	PCIe traffic	detect fallback / unexpected path

• • • mfu

MFU = useful model FLOPs/s ÷ precision-matched peak FLOPs/s

H100 SXM5 REFERENCE

FP8 dense ≈ 1,979 TFLOPS · FP16/BF16 dense ≈ 989 TFLOPS. Do not budget against sparse peak by default.

High SM activity can be synchronization overhead wearing a utilization badge.

CODE → observability/dcgm-prometheus-rules.yaml · mfu-notes.md

A BENCHMARK CLAIM NEEDS A REPRODUCIBLE PLACEMENT MATRIX

CONFIGURATION	PLACEMENT	MEASURE
8 GPU / 1 node	same NVSwitch	algorithm BW · bus BW
16 GPU / 2 nodes	same leaf	message-size sweep
32 GPU / 4 nodes	same leaf	step time · tokens/s · MFU
32 GPU / 4 nodes	cross leaf	same software + message sizes

nccl-tests · single-node baseline

```
./build/all_reduce_perf \
  -b 8M -e 8G -f 2 -g 8
```

CONTROL

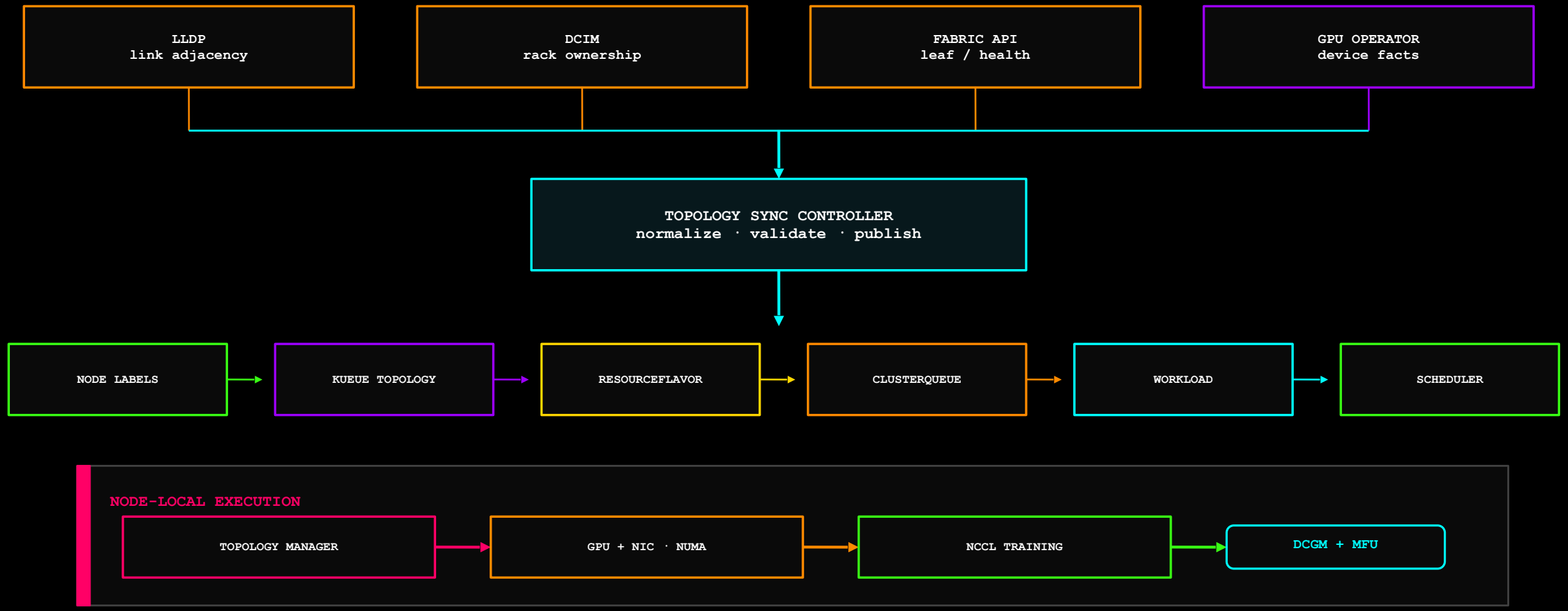
same image · NCCL version · clocks · precision · buckets · MPI/Slurm host mapping

CAPTURE

topologyAssignment · node names · link counters · congestion window

CODE → benchmarks/benchmark-template.md · results-template.csv

THE PRODUCTION CONTROL PLANE — END TO END



Placement quality is designed, admitted, executed and measured—not assumed.

GUIDE → workshop-walkthrough.ipynb · README.md

PLACEMENT QUALITY IS A RESOURCE.

01 **GPUs are not fungible.**

02 Expose physical locality through an owned, versioned contract.

03 Admit the whole distributed workload before consuming GPUs.

04 Solve cluster topology and node-local NUMA with different owners.

05 Measure useful model progress—not merely allocated GPUs.

YOU BUILT a topology inventory · fake GPU capacity · Kueue TAS · required/preferred policy · automated verification

THANK YOU — QUESTIONS.

`make checkpoint-3`

Neeraj Pandey · Co-founder & CTO, Lyntcube